

Assignment 1: Building xv6 Instructor:

Dr. Probir Roy

Part 1 Due Time: 10PM, 9/12/2025

Part 2 Due Time: 10PM, 09/26/2025

xv6 is a port of a classic version of UNIX to a modern processor, Intel's x86. It is a clean and beautiful little kernel. The following link provides a basic introduction to xv6: <https://pdos.csail.mit.edu/6.828/2018/xv6.html>. In this project, we'll learn how to build a xv6 kernel.

1 Part 1

xv6 compiles using the GNU C compiler, targeted at the x86 using ELF binaries. xv6 can boot on real hardware, but typically we run it using the QEMU emulator. We will install a xv6 Linux docker image including the QEMU emulator, and then build xv6 kernel and run it on the QEMU emulator.

1.1 Installing a docker

Visit <https://hub.docker.com/search/?type=edition&offering=community>. Then follow the instruction to download and install a docker on your machine (either Windows, macOS). For Windows users, if your OS is not Windows 10 Professional or Enterprise 64-bit, please download the Docker Toolbox (<https://docs.docker.com/toolbox>).

1.2 Generating an xv6 docker process

I have created an xv6 container image `shqwang/xv6` and stored it in my DockHub repository. If you have a local username, e.g., `foobar`, you can run the following command to generate a container process called `xv6-projects` in your local machine as this (`prompt>` is not a part of the command script):

```
prompt> docker run --name xv6-projects -v /Users/foobar/xv6:/xv6 -it shqwang/xv6 bash
```

Note: For Windows OS, you should run the command as follows:

```
prompt> docker run --name xv6-projects -v /c/Users/foobar/xv6:/xv6 -it shqwang/xv6 bash
```

You shall see the following line at the end of the output:

```
root@4f8a09f17d24:/#
```

It shows that a container process with ID `4f8a09f17d24` has been created. Most likely, you'll see a different ID in your environment. In the above command,

- the option `--name xv6-projects` is used to name the container process as `xv6-projects`;

- the option `-v /Users/foobar/xv6:/xv6` is used to mount a local directory `/Users/foobar/xv6` in the local host to the folder `/xv6` in the container process. You can choose your own local directory under `/Users/foobar`. With this feature, you can build `xv6` inside the container process while editing the source codes in your local machine with any of your favorite editors.
- the option `-it shqwang/xv6 bash` is used to download the contain image and create a container process with a running `bash` shell.

1.3 Building and running xv6

Once the container process is created, it is ready to build `xv6`. Inside the container process, run the following commands (`root@4f8a09f17d24:/#` is not a part of the command script):

```
root@4f8a09f17d24:/# cp /xv6-public/* /xv6/.
root@4f8a09f17d24:/# cd /xv6
root@4f8a09f17d24:/# make
root@4f8a09f17d24:/# make qemu-nox
```

An original source code copy of `xv6` is stored in the folder `/xv6-public`. The first line in the above is to make a copy of the source code to the mounted folder `/xv6`. So you can use the folder `xv6` for any modification while keeping a backup of the original codes under `/xv6-public`. The last two lines are for building and running `xv6`, respectively. Then you shall see something like this:

```
xv6...
cpu1: starting
cpu0: starting
init: starting sh
$
```

You can try the command `ls` in `xv6` as follows:

```
$ ls
```

`xv6` doesn't provide a function to shutdown itself. You can either use the key combination `Ctrl-a x` to shutdown the `xv6` or kill the terminal to shutdown the docker. If you shutdown docker, when you try to access the stopped container process next time, run these:

```
prompt> docker restart xv6-projects
prompt> docker attach xv6-projects
```

You can also use the process ID `4f8a09f17d24` instead of the process name `xv6-projects` in the above commands. Then you shall see this again:

```
root@4f8a09f17d24:/#
```

2 Part 2

2.1 Adding a new user program

Add into `xv6` the first user program called `spin.c`, which is implemented as follows:

```

#include "types.h"
#include "user.h"

int
main(int argc, char *argv[])
{
    int i;
    int x = 0;

    if(argc != 2)
        exit();

    for(i=1; i<atoi(argv[1]); i++)
        x++;

    printf(1, "pid(%d): x = %d\n", getpid(), x);
    exit();
}

```

You can modify it under your local directory (e.g., /Users/foobar/xv6). Any change will be reflected under /xv6 inside the container process. Add into Makefile this user program under UPROGS as

```

UPROGS=\
    _spin\
    _cat\
    ...

```

Make sure to use tab space instead of blank space in front of `_spin`.

Then follow the instruction in **Building and running xv6** to rebuild and rerun the xv6. Then you can test the new user program `spin` in the xv6 shell as follows:

```

init: starting sh
$ spin 10000 & spin 20000 &

```

It basically runs two `spin` processes concurrently in the background. You can change the number 10000 and 20000 to different integers. Try a few different scenarios. Don't miss the symbol ";" in the command, which is different from the macOS command environment I've tested in class.

2.2 Modifying and accessing PCB

In the following, we aim to assign queue information to each process.

Modifying: Under /xv6 directory, you can find `proc.h`, where `struct proc` for any PCB is defined. Add two additional members in `struct proc`: `int queue` and `int quantumsize`.

Initializing: For every newly-created process, the initialization should be performed by `allocproc()` in `proc.c`. For example, `p->state = EMBRYO`. Set `p->queue=0` and `p->quantumsize=4`.

Retrieving: In `scheduler()` of `proc.c`, after `p->state = RUNNING`, use `cprintf()` to print out the queue information of the selected process `p` in terms of `queuetype` and `quantumsize`. `cprintf()` is a debugging function for the kernel. You can find its syntax in some of its occurrences in `proc.c`.

Rebuild `xv6` and rerun

```
$ spin 10000 & spin 20000 &
```

We are expecting to see

```
Process spin is of Process ID 5, Queue Type 0, and Quantum Size 4
...
```

Process ID could be a different number.

3 Resource

If you want to get familiar with the commands of docker, Docker.com provides some good training: <http://training.play-with-docker.com>

Even though in this project you are not required to write any new C code, it might be better for you to get familiar with it now. In next project, we will hack the `xv6` and modify the C source codes. You can find C Programming Online Tutorial here <https://www.tutorialspoint.com/cprogramming>. The online textbook also provides a very brief document to familiarize you with the basics of the C programming environment on UNIX systems: <http://pages.cs.wisc.edu/~remzi/OSTEP/lab-tutorial.pdf>

4 Submission

Form a group if you want to work with another student on Canvas. Specify the contributions made by each member in your reports.

Part 1: In your report, please provide two screenshots showing that `xv6` is running. Each screenshot should include your usernames and the current time, which show that you did it by yourselves. The report should be written in a “.docx”, “.doc”, or “.pdf” format. Submit it to the assignment-1.1 folder on Canvas.

Part 2: Please highlight all changes you made. Show the testing result with a screenshot after you rebuild `xv6` and rerun the testing. The report should be written in a “.docx”, “.doc”, or “.pdf” format. Submit it to the assignment-1.2 folder on Canvas.